SYSTEM DESIGN

Consistent Hashing Visual Guide

Distribute keys across nodes so scaling barely moves any data.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The Resizing Problem

Naive hashing breaks when the cluster changes size.

hash(key) mod N

N = 3: keys spread fine. N = 4 after adding a node:

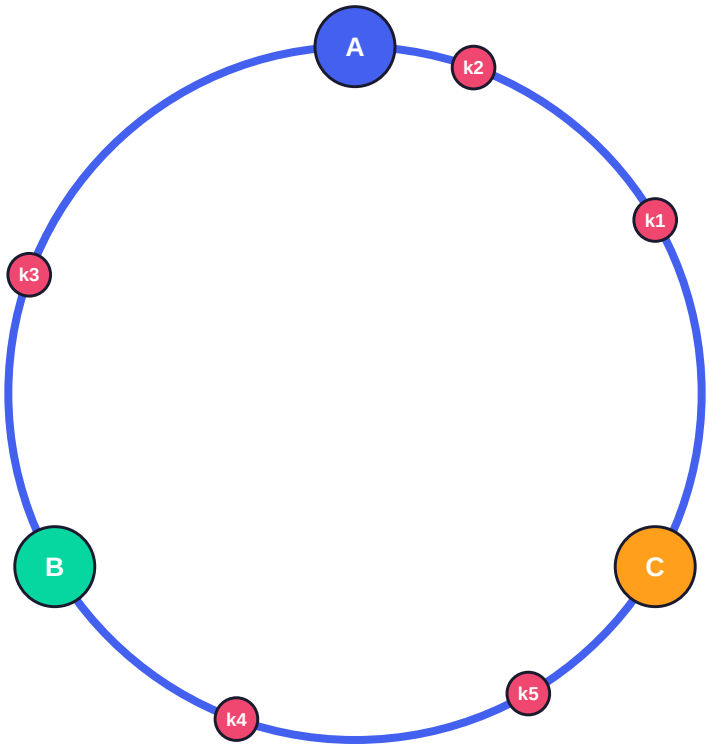
```
key1 -> node 1    key2 -> node 0    key3 -> node 2    (N=3)
key1 -> node 2    key2 -> node 1    key3 -> node 3    (N=4, almost EVERYTHING moves)
```

mod N remaps ~N-1/N of all keys on every resize. At scale that is a full data migration.

STEP 2/6

The Hash Ring

Keys AND nodes live on one ring; keys walk clockwise to their owner.



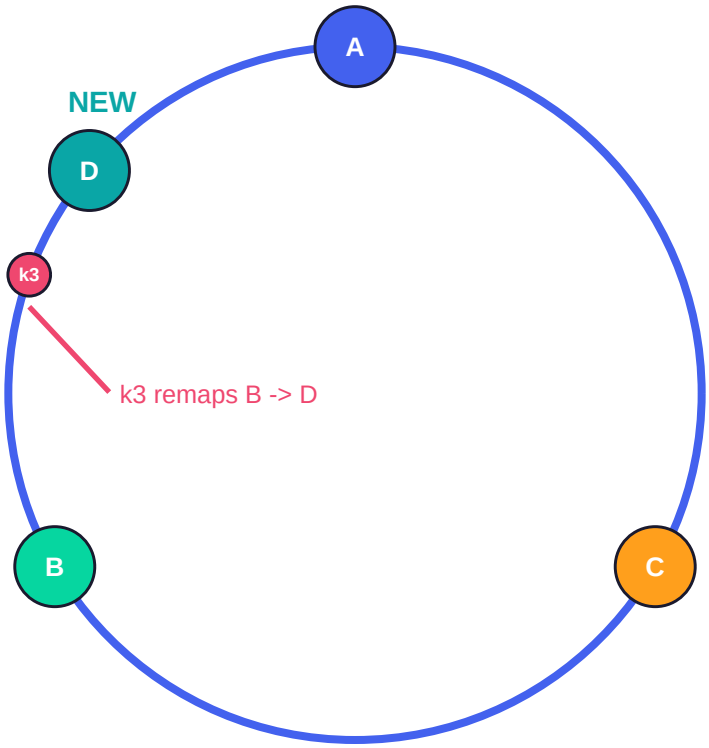
k1 -> A (first node clockwise) k4 -> B k5 -> C

hash(key) and hash(node) share the same space (e.g. 2^32 points on the ring).

STEP 3/6

Adding a Node Moves Little

Only keys between the new node and its predecessor relocate.

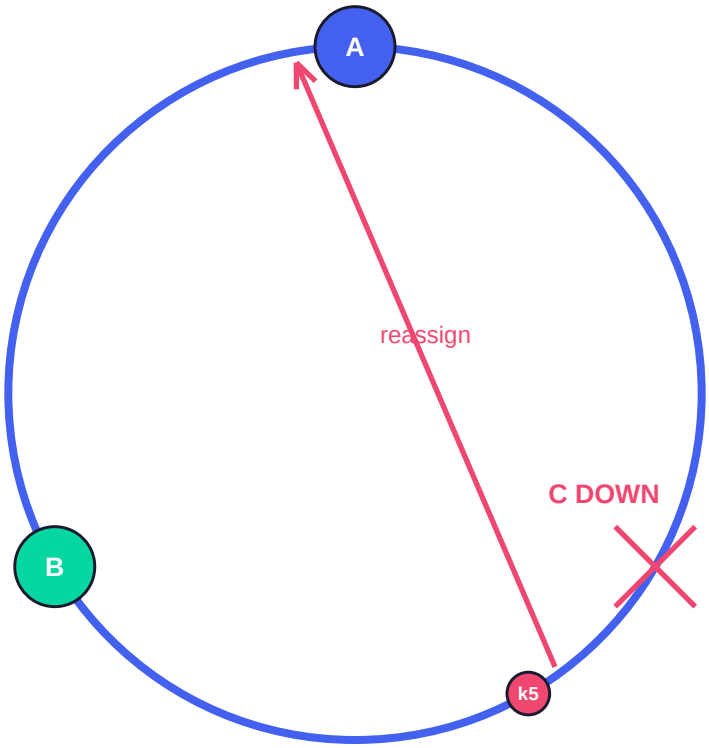


Only ~1/N of keys move per node added. Everyone else's keys stay put. Zero downtime.

STEP 4/6

Removing a Node

Keys fall forward to the next surviving node.

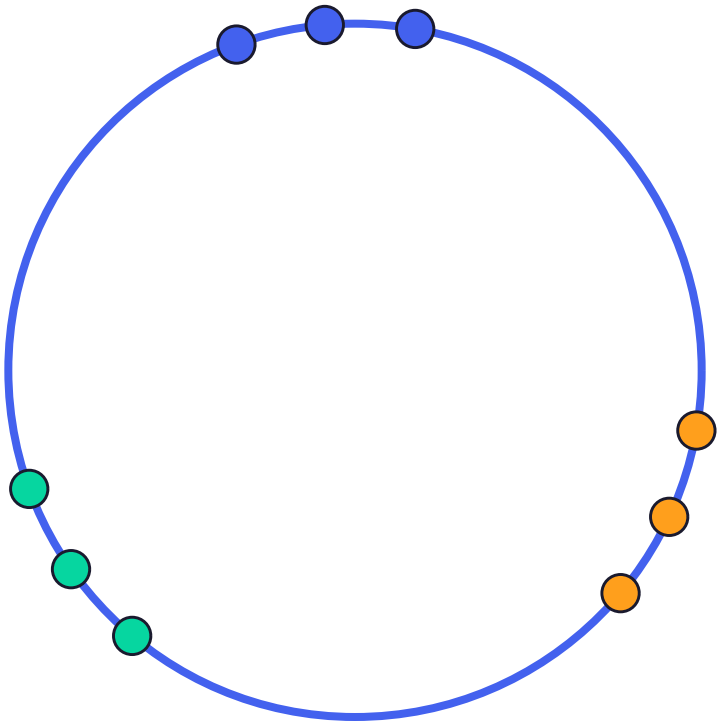


k5's next-clockwise node is gone, so it walks on to A. With replication (RF=3), copies shift the same way.

STEP 5/6

Virtual Nodes Fix Balance

Real rings are lumpy: one node can own 3x its share. Vnodes smooth it out.



Each physical node = many virtual points (e.g. 100-200 vnodes)

- Each vnode is an independent range of the ring: load spreads evenly even with few physical nodes.
- Heterogeneous hardware: give bigger machines MORE vnodes.
- Cassandra calls them vnodes; Dynamo used the same trick.

STEP 6/6

Where It Runs in Production

The workhorse of distributed storage.

DynamoDB

key -> partition via MD5 ring

Cassandra

vnodes (default 16/node), RF copies

Memcached clients

consistent hashing picks the server

CDNs

URL -> edge PoP on the ring

Bonus: the ring also shards requests, not just storage -- any client can compute the owner locally.

STEP ✓/✓

Key Takeaways

Consistent Hashing — Remember

- mod N hashing remaps ~everything on resize; consistent hashing remaps only ~1/N keys.
- Keys and nodes share one hash ring; each key walks clockwise to its owner node.
- Adding/removing a node disturbs only the key range adjacent to it.
- Virtual nodes (many ring points per machine) fix uneven load and honor hardware size.
- DynamoDB, Cassandra, memcached clients, and CDNs all use it to place data and requests.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev